

Connect to Google Drive and Import Packages

```
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

import os

# Replace the path the actual folder name
os.chdir('/content/drive/MyDrive/DS0 579 Advance Sports Analytics/HW1 Premier League Possession')

# Confirm that the files are accessible
os.listdir()

['combined_pl_player_possession.csv',
'combined_pl_squad_possession.csv',
'combined_pl_player_standard_stats.csv',
'combined_pl_squad_standard_stats.csv',
'Premier League Possession & Stats Data 1718-2425.ipynb',
'merged_player_data.csv',
'merged_squad_data.csv',
'Data Set Clean & EDA.ipynb',
'Analysis.ipynb']

import warnings
from scipy.optimize._linesearch import LineSearchWarning # Import the warning class
warnings.filterwarnings("ignore", category=LineSearchWarning)

import scipy.stats as st
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy.stats import t
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LassoCV, RidgeCV
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_predict, LeaveOneOut
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import LogisticRegressionCV, LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc, accuracy_score
from sklearn.metrics import precision_score, recall_score, f1_score, roc_auc_score
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler
import itertools
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.model_selection import GridSearchCV
from sklearn.tree import export_text

pd.set_option('display.max_columns', None)
```

Read & Clean Player Possession Dataset

```
player_df = pd.read_csv('merged_player_data.csv')
player_df

Player Nation Pos is_gk is_df is_mf is_fw Season Squad Age Starts Min 90s xG.1 xAG.1 xG+xAG Touches Def Pen Def 3rd Mid 3rd Att 3rd Att Pen Live At
0 Max Aarons ENG DF 0 1 0 0 2024-2025 Bournemouth 25 1 86 1.0 0.00 0.00 0.00 73.00 2.00 19.00 40.00 15.00 0.00 73.00 2.0
1 Joshua Acheampong ENG DF 0 1 0 0 2024-2025 Chelsea 18 2 170 1.9 0.12 0.00 0.12 72.11 3.68 25.26 42.63 4.21 1.58 72.11 1.0
2 Tyler Adams USA MF 0 0 1 0 2024-2025 Bournemouth 26 13 1290 14.3 0.08 0.06 0.14 61.82 2.59 15.73 35.73 10.91 0.84 61.82 0.9
3 Tosin Adarabioyo ENG DF 0 1 0 0 2024-2025 Chelsea 27 11 1091 12.1 0.07 0.02 0.09 87.77 11.32 42.07 44.21 1.57 0.99 87.77 0.1
4 Simon Adingra CIV FW,MF 0 0 1 1 2024-2025 Brighton 23 7 715 7.9 0.21 0.18 0.39 44.81 0.76 6.58 14.05 24.81 5.70 44.81 4.1
... ... ... ... ... ... ... ... ... ...
4338 Wilfried Zaha CIV FW,MF 0 0 1 1 2017-2018 Crystal Palace 24 28 2549 28.3 0.33 0.17 0.49 40.42 0.18 2.72 13.60 25.62 6.43 40.42 6.9
4339 Davide Zappacosta ITA DF,MF 0 1 1 0 2017-2018 Chelsea 25 12 1098 12.2 0.11 0.11 0.22 68.20 1.64 18.61 20.82 29.75 3.69 68.20 3.0
4340 Marvin Zeegelaar NED DF 0 1 0 0 2017-2018 Watford 26 12 994 11.0 0.00 0.09 0.09 62.09 3.18 16.45 28.82 17.36 1.09 62.09 1.8
4341 Oleksandr Zinchenko UKR DF 0 1 0 0 2017-2018 Manchester City 20 6 532 5.9 0.02 0.07 0.09 131.53 1.69 16.44 81.86 33.39 0.34 131.53 0.5
4342 Kurt Zouma FRA DF 0 1 0 0 2017-2018 Stoke City 22 32 2888 32.1 0.04 0.01 0.05 50.25 7.85 25.02 22.43 3.24 0.69 50.25 0.8

4343 rows x 38 columns

# Replace all inf and -inf values in the DataFrame with NaN
player_df.replace([np.inf, -np.inf], np.nan, inplace=True)

# Optionally, check for missing values now
print("Missing values after replacing inf:\n", player_df.isnull().sum())

Missing values after replacing inf:
Player 0
Nation 2
Pos 0
is_gk 0
is_df 0
is_mf 0
is_fw 0
Season 0
Squad 0
Age 0
Starts 0
Min 0
90s 0
xG.1 2
xAG.1 2
xG+xAG 2
```

```
Touches      84
Def Pen      84
Def 3rd      84
Mid 3rd      84
Att 3rd      84
Att Pen      84
Live         84
Att          84
Succ         84
Succ%        658
Tkld         84
Tkld%        658
Carries     84
TotDist      84
PrgDist      84
PrgC         84
1/3          84
CPA          84
Mis          84
Dis          84
Rec          84
PrgR         84
dtype: int64
```

```
player_df.isnull().sum()
```

	0
Player	0
Nation	2
Pos	0
is_gk	0
is_df	0
is_mf	0
is_fw	0
Season	0
Squad	0
Age	0
Starts	0
Min	0
90s	0
xG.1	2
xAG.1	2
xG+xAG	2
Touches	84
Def Pen	84
Def 3rd	84
Mid 3rd	84
Att 3rd	84
Att Pen	84
Live	84
Att	84
Succ	84
Succ%	658
Tkld	84
Tkld%	658
Carries	84
TotDist	84
PrgDist	84
PrgC	84
1/3	84
CPA	84
Mis	84
Dis	84
Rec	84
PrgR	84
dtype:	int64

```
player_df.columns = player_df.columns.str.strip().str.lower()
```

```
# Impute low-missingness numeric columns (e.g., xG.1, xAG.1, xG+xAG)
low_missing_cols = ['xg.1', 'xag.1', 'xg+xag'] # adjust as needed
for col in low_missing_cols:
    player_df[col] = player_df[col].fillna(player_df[col].median())

# Impute mode to nation column
player_df['nation'] = player_df['nation'].fillna(player_df['nation'].mode()[0])

# For moderate missingness columns like "succ%" and "tkld%", you can impute with the median
for col in ['succ%', 'tkld%']:
    # Optionally create a missing indicator first:
    player_df[col + '_missing'] = player_df[col].isnull().astype(int)
    # Then fill with median:
    player_df[col] = player_df[col].fillna(player_df[col].median())

# For the rest of the numeric columns, if they have a low level of missingness, impute similarly
numeric_cols = ['touches', 'def pen', 'def 3rd', 'mid 3rd', 'att 3rd',
                'att pen', 'live', 'att', 'succ', 'tkld', 'carries', 'totdist',
                'prgdist', 'prgc', '1/3', 'cpa', 'mis', 'dis', 'rec', 'prgr']

for col in numeric_cols:
    player_df[col] = player_df[col].fillna(player_df[col].median())

# Check that there are no more missing values (or that missingness is now handled)
print(player_df.isnull().sum())
```

```
player      0
nation      0
pos         0
is_gk       0
is_df       0
is_mf       0
is_fw       0
season      0
squad       0
age         0
starts      0
min         0
90s         0
xg.1        0
xag.1       0
xg+xag      0
touches     0
def pen     0
def 3rd     0
mid 3rd     0
att 3rd     0
att pen     0
live        0
att         0
succ        0
succ%       0
tkld        0
tkld%       0
carries    0
totdist     0
prgdist     0
prgc        0
1/3         0
cpa         0
mis         0
dis         0
rec         0
prgr        0
succ%_missing 0
tkld%_missing 0
dtype: int64
```

```
# Display data types and null counts for each column
info_df = pd.DataFrame({'dtype': player_df.dtypes, 'nulls': player_df.isnull().sum()})
info_df
```

	dtype	nulls	
player	object	0	
nation	object	0	
pos	object	0	
is_gk	int64	0	
is_df	int64	0	
is_mf	int64	0	
is_fw	int64	0	
season	object	0	
squad	object	0	
age	int64	0	
starts	int64	0	
min	int64	0	
90s	float64	0	
xg.1	float64	0	
xag.1	float64	0	
xg+xag	float64	0	
touches	float64	0	
def pen	float64	0	
def 3rd	float64	0	
mid 3rd	float64	0	
att 3rd	float64	0	
att pen	float64	0	
live	float64	0	
att	float64	0	
succ	float64	0	
succ%	float64	0	
tkld	float64	0	
tkld%	float64	0	
carries	float64	0	
totdist	float64	0	
prgdist	float64	0	
prgc	float64	0	
1/3	float64	0	
cpa	float64	0	
mis	float64	0	
dis	float64	0	
rec	float64	0	
prgr	float64	0	
succ%_missing	int64	0	
tkld%_missing	int64	0	

Next steps:

[Generate code with info_df](#)

[View recommended plots](#)

[New interactive sheet](#)

Do the Validation Set Method on Player Possession Dataset with Linear Regression

```
# Decide which columns to drop from predictors:
# We drop identifiers and any columns directly related to the target.
# For example: 'player', 'nation', 'pos', 'season', 'squad', 'xg.1', 'xa.1', 'xg+xag', and 'xg_xag'
```

```
cols_to_exclude = ['player', 'nation', 'pos', 'season', 'squad', 'age',
                  'starts', 'min', '90s', 'xg.1', 'xag.1', 'xg+xag']

# Select the remaining numeric columns as features:
X = player_df.drop(columns=cols_to_exclude)
y = player_df['xg+xag']

# Split the dataset into training and test sets (e.g., 75/25 split)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# Initialize and train a linear regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)

# Evaluate performance using Mean Squared Error (MSE) and R-squared (R^2)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Validation Set Approach (Linear Regression):")
print("Mean Squared Error (MSE):", mse)
print("R-squared (R^2):", r2)
```

↗ Validation Set Approach (Linear Regression):
Mean Squared Error (MSE): 0.04414880173618489
R-squared (R²): 0.3643454065014976

```
# Add a constant (intercept) term to X_train
X_train_const = sm.add_constant(X_train)

# Fit the model using OLS
model_sm = sm.OLS(y_train, X_train_const).fit()

# Print the full summary table
print(model_sm.summary())
```

↗

OLS Regression Results						
=====						
Dep. Variable:	xg+xag	R-squared:	0.364			
Model:	OLS	Adj. R-squared:	0.359			
Method:	Least Squares	F-statistic:	68.54			
Date:	Fri, 04 Apr 2025	Prob (F-statistic):	5.40e-293			
Time:	05:10:50	Log-Likelihood:	-573.72			
No. Observations:	3257	AIC:	1203.			
Df Residuals:	3229	BIC:	1374.			
Df Model:	27					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	0.1644	0.027	5.982	0.000	0.111	0.218
is_gk	-0.1433	0.060	-2.391	0.017	-0.261	-0.026
is_df	-0.0591	0.021	-2.826	0.005	-0.100	-0.018
is_mf	-0.0280	0.015	-1.861	0.063	-0.057	0.001
is_fw	0.0293	0.019	1.545	0.122	-0.008	0.067
touches	1.1392	0.153	7.441	0.000	0.839	1.439
def pen	0.0028	0.003	1.086	0.277	-0.002	0.008
def 3rd	0.0194	0.010	1.888	0.059	-0.001	0.040
mid 3rd	0.0236	0.010	2.324	0.020	0.004	0.044
att 3rd	0.0239	0.010	2.314	0.021	0.004	0.044
att pen	0.0527	0.004	15.060	0.000	0.046	0.060
live	-1.1637	0.153	-7.616	0.000	-1.463	-0.864
att	-0.0103	0.012	-0.844	0.399	-0.034	0.014
succ	-0.0050	0.014	-0.365	0.715	-0.032	0.022
succ%	0.0015	0.000	8.695	0.000	0.001	0.002
tkld	0.0055	0.017	0.329	0.742	-0.027	0.038
tkld%	1.355e-05	0.000	0.103	0.918	-0.000	0.000
carries	0.0041	0.002	2.543	0.011	0.001	0.007
totdist	-0.0008	0.000	-3.667	0.000	-0.001	-0.000
prgdist	0.0010	0.000	3.465	0.001	0.000	0.002
prgc	-0.0107	0.006	-1.697	0.090	-0.023	0.002
1/3	-0.0091	0.006	-1.553	0.121	-0.021	0.002
cpa	0.0038	0.010	0.377	0.706	-0.016	0.023
mis	-0.0261	0.004	-6.952	0.000	-0.033	-0.019
dis	-0.0017	0.004	-0.422	0.673	-0.009	0.006
rec	-0.0003	0.001	-0.189	0.850	-0.003	0.003
prgr	0.0149	0.003	5.364	0.000	0.009	0.020
succ%_missing	0.0121	0.009	1.375	0.169	-0.005	0.029
tkld%_missing	0.0121	0.009	1.375	0.169	-0.005	0.029
=====						
Omnibus:	6491.326	Durbin-Watson:	2.028			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	22696685.231			
Skew:	15.758	Prob(JB):	0.00			
Kurtosis:	410.741	Cond. No.	1.11e+16			
=====						

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The smallest eigenvalue is 1.46e-24. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

✓ Do the K-fold Method on Player Possession with Logistic Regression

```
# Create a binary target: 1 if xg_xag is above the median, 0 otherwise.
median_val = player_df['xg+xag'].median()
player_df['xg+xag_binary'] = (player_df['xg+xag'] > median_val).astype(int)

# Define predictors by dropping identification and target-related columns.
cols_to_exclude = ['player', 'nation', 'pos', 'season', 'squad', 'age',
                  'starts', 'min', '90s', 'xg.1', 'xag.1', 'xg+xag', 'xg+xag_binary']

X = player_df.drop(columns=cols_to_exclude)
y = player_df['xg+xag_binary']

# Replace infinite values with NaN and fill missing values with the median.
X.replace([np.inf, -np.inf], np.nan, inplace=True)
X = X.fillna(X.median())

# Standardize (scale) the features.
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Set up Stratified K-Fold Cross-Validation (5 splits).
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Initialize the logistic regression model with increased max_iter and solver 'saga'
log_model = LogisticRegression(solver='saga', max_iter=5000, random_state=42)

# Use cross_val_predict to generate cross-validated predictions
y_pred_cv = cross_val_predict(log_model, X_scaled, y, cv=cv)

# Compute the confusion matrix and accuracy
cm = confusion_matrix(y, y_pred_cv)
```

```
cm = confusion_matrix(y, y_pred_cv)
accuracy = accuracy_score(y, y_pred_cv)

print("K-Fold Cross-Validation (Stratified 5 splits) Logistic Regression:")
print("Confusion Matrix:")
print(cm)
print("Accuracy:", accuracy)
```

↗ K-Fold Cross-Validation (Stratified 5 splits) Logistic Regression:
Confusion Matrix:
[[2038 235]
 [346 1724]]
Accuracy: 0.8662215058715174

```
# Assuming y and y_pred_cv are your true labels and predicted labels from cross-validation.
precision = precision_score(y, y_pred_cv)
recall = recall_score(y, y_pred_cv)
f1 = f1_score(y, y_pred_cv)
auc = roc_auc_score(y, y_pred_cv)

print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("ROC-AUC:", auc)
```

↗ Precision: 0.8800408371618172
Recall: 0.8328502415458937
F1 Score: 0.855795482750062
ROC-AUC: 0.8647313240285561

⌵ Read & Clean Squad Possession Dataset

```
squad_df = pd.read_csv('merged_squad_data.csv')
squad_df
```

↗

	Squad	Season	Poss	Age	xG.1	xAG.1	xG+xAG	Touches	Def Pen	Def 3rd	Mid 3rd	Att 3rd	Att Pen	Live	Att	Succ	Succ%	Tkld	Tkld%	Carries	TotDist	PrgDist	PrgC	1
0	Arsenal	2024-2025	55.7	26.5	1.56	1.19	2.75	623.77	47.70	158.50	270.20	199.90	33.07	623.70	19.50	8.50	1.45	8.60	1.47	362.97	1920.73	1015.07	22.13	13
1	Aston Villa	2024-2025	51.2	27.8	1.41	1.05	2.46	579.27	68.77	196.63	248.40	139.90	24.87	579.17	19.63	8.87	1.51	8.20	1.39	327.03	1799.07	941.43	18.90	12
2	Bournemouth	2024-2025	47.6	25.8	1.83	1.24	3.07	565.83	58.17	169.27	241.87	161.97	27.70	565.60	18.70	8.00	1.43	8.57	1.53	303.60	1732.17	921.20	19.50	13
3	Brentford	2024-2025	48.3	26.6	1.54	1.11	2.65	576.50	88.73	223.47	214.87	143.20	26.03	576.33	16.60	6.97	1.40	7.80	1.57	305.10	1652.97	837.80	16.03	9
4	Brighton	2024-2025	51.9	25.6	1.49	1.05	2.54	618.50	69.67	201.27	268.27	155.93	27.43	618.40	21.97	9.17	1.39	10.70	1.62	354.63	1983.23	1085.07	21.17	14
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
155	Swansea City	2017-2018	45.3	27.2	0.83	0.53	1.36	559.95	64.24	185.45	260.74	119.84	14.11	559.92	13.76	8.37	1.60	5.39	1.03	279.29	1631.79	834.66	15.68	11
156	Tottenham	2017-2018	61.8	25.8	1.70	1.29	2.99	732.13	54.74	188.34	354.37	196.47	24.97	732.05	18.92	11.47	1.59	7.39	1.03	388.61	2125.18	1112.08	22.68	16
157	Watford	2017-2018	49.7	27.9	1.19	0.77	1.95	570.84	54.92	166.26	269.08	141.50	20.61	570.68	14.87	8.74	1.55	6.13	1.08	310.82	1537.55	778.79	17.03	12
158	West Brom	2017-2018	40.7	29.1	0.96	0.68	1.64	504.84	60.58	176.26	216.34	117.58	15.42	504.82	13.87	8.45	1.60	5.39	1.02	258.42	1426.87	704.03	13.34	10
159	West Ham	2017-2018	45.2	28.0	0.97	0.65	1.62	538.26	59.61	163.11	259.32	122.29	15.55	538.16	16.89	11.13	1.73	5.74	0.89	265.42	1599.89	735.82	13.76	11

160 rows x 29 columns

```
# Replace all inf and -inf values in the DataFrame with NaN
squad_df.replace([np.inf, -np.inf], np.nan, inplace=True)

# Optionally, check for missing values now
print("Missing values after replacing inf:\n", squad_df.isnull().sum())
```

↗ Missing values after replacing inf:
Squad 0
Season 0
Poss 0
Age 0
xG.1 0
xAG.1 0
xG+xAG 0
Touches 0
Def Pen 0
Def 3rd 0
Mid 3rd 0
Att 3rd 0
Att Pen 0
Live 0
Att 0
Succ 0
Succ% 0
Tkld 0
Tkld% 0
Carries 0
TotDist 0
PrgDist 0
PrgC 0
1/3 0
CPA 0
Mis 0
Dis 0
Rec 0
PrgR 0
dtype: int64

```
squad_df.columns = squad_df.columns.str.strip().str.lower()
```

⌵ Do the LOOCV Method on Squad Possession with Multi-Logistic Regression

```
# Create a multi-class target based on xg_xag
# For example, split xg_xag into 3 classes (low, medium, high) using quantiles
squad_df['performance_class'] = pd.qcut(squad_df['xg+xag'], q=3, labels=[0, 1, 2]).astype(int)

# Check the distribution of the new target variable
print("Performance Class Distribution:")
print(squad_df['performance_class'].value_counts())

# Define predictor columns by dropping identifiers and target-related columns.
```

```
# Here we drop: 'squad', 'season', 'xg.1', 'xa.1', 'xg+xag', 'xg_xag', and 'performance_class'
cols_to_exclude = ['squad', 'season', 'age', 'xg.1', 'xag.1', 'xg+xag', 'performance_class']
X = squad_df.drop(columns=cols_to_exclude)
y = squad_df['performance_class']

# Scale the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Set up Leave-One-Out Cross-Validation
loo = LeaveOneOut()

# Initialize a multinomial logistic regression model.
# We use multi_class='multinomial' and a high max_iter to help convergence.
model = LogisticRegression(solver='saga', max_iter=50000, random_state=42)

# Use cross_val_predict with L00CV to generate predictions for each observation
y_pred_loo = cross_val_predict(model, X_scaled, y, cv=loo)

# Evaluate performance with a confusion matrix, accuracy, and a classification report.
cm = confusion_matrix(y, y_pred_loo)
accuracy = accuracy_score(y, y_pred_loo)
report = classification_report(y, y_pred_loo)

print("\nL00CV Multi-class Logistic Regression on Squad Data (target = performance_class from xg_xag):")
print("Confusion Matrix:")
print(cm)
print("Accuracy:", accuracy)
print("Classification Report:")
print(report)
```

 Performance Class Distribution:

```
performance_class
2      54
1      53
0      53
Name: count, dtype: int64
```

L00CV Multi-class Logistic Regression on Squad Data (target = performance_class from xg_xag):

Confusion Matrix:

```
[[41 12  0]
 [11 35  7]
 [ 0 10 44]]
```

Accuracy: 0.75

Classification Report:

	precision	recall	f1-score	support
0	0.79	0.77	0.78	53
1	0.61	0.66	0.64	53
2	0.86	0.81	0.84	54
accuracy			0.75	160
macro avg	0.76	0.75	0.75	160
weighted avg	0.76	0.75	0.75	160